Расчетно-графическая работа

по дисциплине «Структуры и алгоритмы обработки данных»
направление 09.03.01 «Информатика и вычислительная техника
направление 09.03.04 «Программная инженерия»
3 курс, 5 семестр

Цель РГР – освоение технологии разработки комплексного программного обеспечения для решения задач в различных прикладных областях.

Задание 1 2
Задание 2 5
Задание 3 6
Методические указания. 8
Содержание отчета 9
Список рекомендуемой литературы 10

Задание 1

1) Спроектировать и реализовать шаблонный класс для коллекции «**Простой** граф» и использовать коллекцию для решения задач для неориентированных, ориентированных и взвешенных графов.

Разработать АТД «Простой граф».

Интерфейс АТД «Простой граф» включает операции:

Конструктор ( ) по умолчанию: создает пустой L - граф с нулевым числом вершин и ребер,
Конструктор (V, D, F) создает граф с **V** вершинами, без ребер, типа **D** (ориентированный / неориентированный), формы представления **F** (L- граф/M-граф),
Конструктор (V, E, D, F) создает граф с **V** вершинами, с **E** случайными ребрами, типа **D** (ориентированный / неориентированный), формы представления **F** (L- граф/M-граф),
Конструктор (G) - конструктор копирования создает объект – копию графа **G,**
Деструктор ( ) уничтожает внутренние структуры графа,
V ( ) - возвращает число вершин в графе,
E ( ) - возвращает число ребер в графе**,**
Directed ( ) - возвращает тип графа (ориентированный / неориентированный)
Dense ( ) - возвращает форму представления графа (L- граф / M- граф),
K ( ) - возвращает коэффициент насыщенности графа,
ToListGraph ( ) преобразует граф к L- графу,
ToMatrixGraph ( ) преобразует граф к M- графу,
InsertV ( ) добавляет безымянную вершину к графу и возвращает адрес дескриптора вновь созданной вершины
InsertV (*name*) добавляет вершину с именем ***name*** к графу и возвращает адрес дескриптора вновь созданной вершины,
DeleteV (*v*) - удаляет вершину из графа, заданную адресом дескриптора ***v***,
InsertE(*v1, v2*) - добавляет ребро между вершинами графа, заданными адресами дескрипторов ***v1*** и ***v2*** и возвращает адрес дескриптора вновь созданного ребра.
InsertE(*v1, v2, w*) - добавляет ребро между вершинами графа, заданными адресами дескрипторов ***v1*** и ***v2,*** с весом ***w*** и возвращает адрес дескриптора вновь созданного ребра.
DeleteE (*v1, v2*) удаляет ребро, соединяющее вершины, заданные адресами дескрипторов ***v1*** и ***v2,***
GetEdge (*v1, v2*) возвращает адрес дескриптора ребра соединяющего вершины, заданные дескрипторами ***v1*** и ***v2***,

Разработать ассоциированные с графом объекты:

«Дескриптор вершины графа»

Дескриптор вершины содержит поля:
name **–** имя вершины,
data **–** данные, связанные с вершиной,

Интерфейс АТД «Дескриптор вершины графа» включает операции:
Конструктор ():поле ***name*** не определено,
Конструктор (***name***): ***name*** - имя вершины,
SetName (*name*) – задает имя вершины,
SetData (*data*) – записывает данные ***data*** в дескриптор вершины.
GetName ( ) - возвращает имя вершины,
GetData ( ) - возвращает данные, связанные с вершиной,

«Дескриптор ребра графа»

Дескриптор ребра содержит поля:
v1 - дескриптор вершины, из которой исходит ребро,
v2 - дескриптор вершины, в которую входит ребро,
w - вес ребра,
data - данные, связанные с ребром,

Интерфейс АТД «Дескриптор ребра графа» включает операции:
Конструктор (***v1, v2***): ***v1*** - дескриптор вершины, из которой исходит ребро, ***v2*** - дескриптор вершины, в которую входит ребро,
Конструктор (***v1, v2, w***): ***v1*** - дескриптор вершины, из которой исходит ребро, ***v2*** - дескриптор вершины, в которую входит ребро, ***w*** - вес ребра,

v1( ) - возвращает дескриптор вершины, из которой исходит ребро,
v2( ) - возвращает дескриптор вершины, в которую входит ребро,
SetW (*w*) - задание веса ребра,
SetData (***data***) - изменение данных, связанных с ребром,
GetW ( ) - возвращает вес ребра,
GetData ( ) - возвращает данные, связанные с ребром.
.

АТД «Итератор вершин графа»

Дескриптор итератора содержит поля:

g **–** указатель на граф, к которому привязан итератор,
v - дескриптор вершины в графе, на которую указывает итератор.

Интерфейс АТД «Итератор вершин графа» включает операции:
Конструктор (***g***) - создает итератор, установленный на дескриптор первой вершины в структуре графа ***g,***
beg ( ) – устанавливает итератор на дескриптор первой вершины в структуре графа ***g***,
end ( ) - возвращает признак окончания переходов итератора,
operator ++ - переход на следующую вершину в структуре графа,
operator * - возвращает дескриптор вершины ***v***, на которую указывает итератор.

АТД «Итератор ребер графа»

Дескриптор итератора содержит поля:

g **–** указатель на граф, к которому привязан итератор,
e - дескриптор ребра в графе, на которое установлен итератор,.

Интерфейс АТД «Итератор ребер графа» включает операции:
Конструктор (***g***) - создает итератор, установленный на первое ребро в структуре графа
beg ( ) – устанавливает итератор на дескриптор первого ребра в структуре графа ***g***,
end ( ) - возвращает признак окончания переходов итератора,
operator ++ - переход на следующее ребро в структуре графа,
operator * - возвращает дескриптор ребра ***e***, на которое указывает итератор.

АТД «Итератор исходящих ребер вершины»

Дескриптор итератора содержит поля:

g **–** указатель на граф, к которому привязан итератор,
v - дескриптор заданной вершины для итератора,
e - дескриптор ребра в графе, на которое установлен итератор.

Интерфейс АТД «Итератор исходящих ребер вершины» включает операции:
Конструктор (***g, v***) - создает итератор исходящих ребер для графа ***g,*** вершины ***v***, установленный на первое исходящее ребро вершины ***v***,
beg ( ) - устанавливает итератор на дескриптор первого исходящего ребра вершины ***v***,
end ( ) - возвращает признак окончания переходов итератора,
operator ++ - переход к следующему исходящему ребру вершины ***v***,
operator * - возвращает дескриптор исходящего ребра ***e***, на которое указывает итератор.

Задание 2

Спроектировать и реализовать шаблонный класс для ***АТД «Задача для невзвешенного графа»*** в соответствии с вариантом и использовать для решения задачи на неориентированном или ориентированном графе. Для программирования графа использовать разработанный в задании 1 - ***АТД «Простой граф».***

Интерфейс ***АТД «Задача для невзвешенного графа»*** включает операции:

Конструктор (***g***) - создает объект задачи, ассоциированный с графом ***g***, и выполняет решение задачи для графа ***g***,
Конструктор (*T*) - конструктор копирования создает копию объекта – задачи ***T*,**
Деструктор ( ) - уничтожает внутренние структуры объекта задачи,
Set (***g***) – связывает объект задачи с графом ***g*** и выполняет решение задачи для графа ***g***,
Restart ( ) – повторно выполняет решение задачи для графа ***g***,
Result ( ) – возвращает результат решения задачи.

Варианты задачи:

1) преобразование неориентированного графа в двудольный с помощью раскраски его вершин в два цвета,
2) формирование кратчайших по числу ребер путей между всеми парами вершин орграфа,
3) определение и формирование гамильтонова цикла в орграфе,
4) определение диаметра связного неориентированного графа,
5) формирование списка вершин в результате прямой топологической сортировки ациклического орграфа DAG (сортировка - на основе очереди истоков),
6) формирование остовного дерева орграфа в пределах заданной высоты за счёт добавления минимального числа рёбер,
7) формирование списка вершин на основе топологической сортировки ациклического орграфа DAG на основе поиска в глубину,
8) поиск и формирование списков для всех простых циклов, включающих заданную вершину орграфа,
9) преобразование неориентированного графа в реберно-связный неориентированный граф,
10) поиск и формирование в орграфе простого цикла заданной длины, включающего заданную вершину,
11) формирование редуцированного орграфа сильносвязных компонент для заданного орграфа,
12) формирование структуры с классификацией всех рёбер относительно заданной вершины орграфа,
13) для неориентированного графа формирование списка вершин, отстоящих на расстоянии ***d*** от заданной вершины (***d*** – число рёбер),
14) преобразование неориентированного графа в двусвязный неориентированный граф,
15) определение и формирование эйлерова цикла в неориентированном графе,
16) формирование списка ребер неориентированного графа в порядке двухпроходного эйлерова цикла.

Задание 3

Спроектировать и реализовать шаблонный класс для ***АТД «Задача для взвешенного графа»»*** в соответствии с вариантом и использовать для решения задачи на взвешенном графе. Для программирования графа использовать разработанный в задании 1- ***АТД «Простой граф».***

Интерфейс ***АТД «Задача для взвешенного графа»»*** включает операции:

Конструктор (g) - создает объект задачи , ассоциированный с графом g, и выполняет решение задачи для графа g,

Конструктор (*T*) - конструктор копирования создает копию объекта – задачи ***T,***

Деструктор ( ) - уничтожает внутренние структуры объекта задачи,

Set (g) – связывает объект задачи с графом g и выполняет решение задачи для графа g,

Restart ( ) – повторно выполняет решение задачи для графа g,

Result( ) – возвращает результат решения задачи.

Варианты задачи:

1) для взвешенного орграфа формирование диаметра и пути, соответствующего диаметру, на основе алгоритма Флойда,

2) преобразование взвешенного орграфа: приведение отрицательной весовой функции к положительной методом Джонсона,

3) для взвешенного орграфа формирование входящих в заданную вершину кратчайших путей на основе алгоритма Дейкстры,

4) формирование множества вершин, входящих в центр взвешенного орграфа на основе алгоритма Беллмана – Форда.

5) формирование для взвешенного неориентированного графа множества кластеров, объединяющих вершины ребрами с длиной, большей d. Для разбиения использовать алгоритм Крускалла,

6) формирование множества вершин, входящих в периферию взвешенного орграфа (использовать алгоритм Дейкстры)

7) формирование списка вершин, которые лежат в пределах заданного расстояния d от заданной вершины взвешенного орграфа с отрицательной весовой функции на основе алгоритма Беллмана-Форда,

8) формирование первого по величине (по количеству вершин) кластера в неориентированном взвешенном графе, вершины которого объединены ребрами с длиной, большей d. Для определения кластера использовать алгоритм Прима,

9 формирование множества вершин, входящих в периферию взвешенного орграфа на основе алгоритма Флойда,

10) нахождения ребра (ребер), устранение которого вызывает максимальное возрастание кратчайшего пути из вершины ***u*** в вершину ***v*** во взвешенном орграфе на основе алгоритма Дейкстры,

11) формирование кратчайших путей для всех пар вершин взвешенного орграфа на основе алгоритма Беллмана-Форда,

12) формирование минимального остовного дерева методом Прима относительно заданной вершины ***u*** взвешенного неориентированного графа,

13) формирование радиуса и соответствующего радиусу пути взвешенного орграфа на основе алгоритма Дейкстры,

14) формирование множества вершин, входящих в центр взвешенного орграфа на основе алгоритма Флойда,

15) определение эксцентриситета заданной вершины взвешенного орграфа с отрицательной весовой функции на основе алгоритма Беллмана-Форда,

16) формирование остовного дерева взвешенного неориентированного графа с суммарным весом ребер, ближайшим к заданному весу d. Для формирования дерева использовать алгоритм Крускалла,

Методические указания.

1) Граф реализуется в виде класса «Простой граф» и ассоциированных классов «Дескриптор вершины графа», «Дескриптор ребра графа», «Итератор вершин графа», «Итератор ребер графа», «Итератор исходящих ребер вершины.

2) Объект «Дескриптор вершины графа» реализуется в виде шаблонного класса. Параметрами шаблона являются тип, задающий имя вершины, и тип данных, связанных с вершиной.

3) Объект «Дескриптор ребра графа» реализуется в виде шаблонного класса. Параметрами шаблона являются тип «Дескриптор вершины графа», тип, задающий вес ребра, и тип данных, связанных с ребром.

4) Объект «Простой граф» реализуется в виде шаблонного класса. Параметрами шаблона являются типы «Дескриптор вершины графа» и «Дескриптор ребра графа».

5) Определения ассоциированных классов («Итератор вершин графа», «Итератор ребер графа», «Итератор исходящих ребер вершины») вложены в определение класса «Простой граф».

6) Для реализации различных форм представления графа используются вспомогательные классы «L – граф» и «M – граф». Рекомендуется объединить эти классы в иерархию с использованием наследования и полиморфизма.

7) Объект «Простой граф» содержит вектор дескрипторов вершин, вставленных в граф.

8) Объект «Простой граф» содержит структуру графа в виде списков смежности (L-граф) или матрицы смежности(M-граф). Элементы списков смежности или матрицы смежности содержат дескрипторы ребер, вставленных в граф.

9) Для согласования имён вершин в прикладной программе с дескрипторами вершин в объекте «Простой граф» рекомендуется использовать объект «Отображение имени на адрес дескриптора вершины». Отображение реализуется в виде шаблонного класса. Параметрами шаблона являются тип имени вершины и тип «Дескриптор вершины графа». Такой объект обеспечивает прямой доступ по имени вершины к адресу дескриптора вершины.

10) Для реализации задач, заданных в вариантах задания 2 и 3, разработать классы – клиенты для АТД «Простой граф».

11) Объект задачи связан с объектом графа, для которого решается задача, с помощью указателя на объект «Простой граф».

12) Для решения задачи объект задачи использует интерфейс объекта «Простой граф» и ассоциированные с графом объекты («Дескриптор вершины графа», «Дескриптор ребра графа», «Итератор вершин графа», «Итератор ребер графа», «Итератор исходящих ребер вершины»).

13) Объект задачи содержит все структуры, необходимые для решения задачи, сохранения результатов решения задачи.

14) Автоматическое решение задачи происходит в момент создания объекта задачи, вызова метода **Restart ( )** (повторное решение задачи для графа), в момент вызова методов **Set (g)** (связывание объекта задачи с другим графом g),.

15) Метод **Result ( )** возвращает результат решения задачи клиентской программе в виде программной структуры, удобной для программной обработки.

16) Разработать консольное приложение для визуализации структуры графа (**V** < **20**). Программа должна обеспечивать визуальный просмотр структуры графа, результатов работы отдельных операций АТД «Простой граф» и ассоциированных АТД («Дескриптор вершины графа», «Дескриптор ребра графа», «Итератор вершин графа», «Итератор ребер графа», «Итератор исходящих ребер вершины»), решения задач согласно заданиям 2,3.
Программа визуализации должна обеспечивать возможность модификации структуры графа (вставка или удаление вершин, ребер, изменение параметров, связанных с вершинами, ребрами) и повторного решения задач.

Содержание отчета

1) Оглавление.
2) Общее задание.
3) Диаграмма взаимосвязи объектов, реализующих АТД «Простой граф», вспомогательных объектов, АТД задач 1 – 2.
4) **Формат АТД «Простой граф».**
5) Интерфейс объекта «Простой граф» в виде определения класса на С++.
6) Интерфейс объекта «Дескриптор вершины» в виде определения класса на С++.
7) Интерфейс объекта «Дескриптор ребра» в виде определения класса на С++.
8) Интерфейс объекта «Итератор вершин» в виде определения класса на С++.
9) Интерфейс объекта «Итератор ребер» в виде определения класса на С++.
10) Интерфейс объекта «Итератор исходящих ребер» в виде определения класса на С++.
11) **Формат АТД «Задача 1» для *Задания2*.**
12) Интерфейс объекта «Задача 1» в виде определения класса на С++.
13) Краткое описание алгоритма, теоретическая оценка трудоемкости задачи 1.
14) Пример работы программы (скрин окна консольного приложения). В окне должны быть отображены структура графа и результат решения задачи 1 на данном графе, сформированный методом Result задачи 1.
15) **Формат АТД «Задача 2» для *Задания3*.**
16) Интерфейс объекта «Задача 2» в виде определения класса на С++.
17) Краткое описание алгоритма, теоретическая оценка трудоемкости задачи 2.
18) Пример работы программы (скрин окна консольного приложения). В окне должны быть отображены структура графа и результат решения задачи 2 на данном графе, сформированный методом Result задачи 2.
19) Заключение.
20) Список использованных источников.
21) приложение с текстами программ (**представить в электронном виде в файле .docx**):
    - тексты программ для всех классов.
    - текст консольного приложения для визуализации структуры графа и работы операций и задач для графа.

Список рекомендуемой литературы

1. Альфред Ахо, Джон Э. Хопкрофт, Д. Ульман Структуры данных и алгоритмы. - М. - СПб – Киев: Вильямс, 2000 г. – 384 с.
2. Ананий Левитин Алгоритмы. Введение в разработку и анализ. – М. – СПб – Киев: Вильямс, 2006 г. – 576 с.
3. Фрэнк М. Каррано, Джанет Дж. Причард. Абстракция данных и решение задач на С++. Стены и зеркала. - М. - СПб – Киев: Вильямс, 2003 г. – 848 с.
4. Коллинз Уильям Дж. Структуры данных и стандартная библиотека шаблонов. – М.: ООО «Бином – Пресс», 2004 г. – 624 с.
5. Т. Кормен, Ч. Лейзерсон, Р. Ривест Алгоритмы. Анализ и построение. - М: БИНОМ, 2000 г. – 960 с.
6. Джерими Сик, Лай-Кван Ли, Эндрю Ламсдэйн. C++ Boost Graph Library. Библиотека программиста. – СПб: Питер. 2006 г. – 304 с.
7. Роберт Сэджвик. Фундаментальные алгоритмы на С++. Часть 5. Алгоритмы на графах - М: DiaSoft, 2002 г.
8. НОУ ИНТУИТ Алгоритмы на С++. Лекция 17: Виды графов и их свойства https://intuit.ru/studies/professional_skill_improvements/17714/courses/1174/lecture/25264?page=1